

Laporan Tugas Besar 2
IF3270 - Pembelajaran Mesin
Convolutional Neural Network dan Recurrent
Neural Network



Disusun oleh:

Kelompok : 2_Karyawan_Tettap

Stanislaus Ardy Bramantyo	18223057
Nazwan Siddqi Muttaqin	18223066
Matthew Sebastian Kurniawan	18223096

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

I. Daftar Isi

I. Daftar Isi.....	2
II. Deskripsi Persoalan.....	4
III. Pembahasan.....	5
A. Penjelasan Implementasi.....	5
1. CNN.....	5
a. Kelas Conv2DLayer.....	5
b. Kelas LocallyConnected2DLayer.....	6
c. Kelas Pooling.....	7
d. Kelas FlattenLayer.....	8
e. Kelas DenseLayer.....	8
f. Kelas BatchNormLayer.....	9
g. Kelas CNNFromScratch.....	10
h. Utilitas Visualisasi (Bonus).....	11
2. Simple RNN & LSTM.....	11
a. Kelas EmbeddingLayer.....	11
b. Kelas SimpleRNNCell.....	12
c. Kelas LSTMCell.....	13
d. Kelas DenseLayer.....	14
e. Kelas CaptioningFromScratch.....	15
B. Penjelasan Forward Propagation.....	17
1. CNN.....	17
a. Convolutional Layer [Conv2DLayer / LocallyConnected2DLayer].....	17
b. Batch Normalization.....	18
c. Pooling Layer.....	18
d. Flatten Layer.....	18
e. Dense Layer.....	18
2. Simple RNN & LSTM.....	19
a. CNN Layer.....	19
b. Dense Layer.....	19
c. Inject Feature.....	19
d. Embedding.....	20
e. RNN/LSTM Forward.....	20
f. Prediction.....	20
C. Hasil Pengujian.....	21
1. CNN.....	21
A) Perbandingan shared vs non-shared parameter.....	21
B) Pengaruh jumlah layer konvolusi.....	22

C) Pengaruh banyak filter per layer konvolusi.....	23
D) Pengaruh ukuran filter per layer konvolusi.....	24
E) Pengaruh jenis pooling layer.....	25
F) Perbandingan Keras vs from scratch.....	26
2. Simple RNN dan LSTM untuk Image Captioning.....	27
A) Perbandingan jumlah layer: RNN.....	27
B) Perbandingan jumlah layer: LSTM.....	28
C) Perbandingan ukuran hidden state: RNN.....	28
D) Perbandingan ukuran hidden state: LSTM.....	28
E) Perbandingan RNN vs LSTM.....	28
F) Perbandingan Keras vs from scratch.....	30
G) Pengaruh panjang maksimum caption terhadap BLEU-4.....	31
IV. Kesimpulan dan Saran.....	33
A. Kesimpulan.....	33
B. Saran.....	33
V. Pembagian Tugas.....	35
VI. Referensi.....	36
VII. Lampiran.....	37

II. Deskripsi Persoalan

Tugas Besar II mata kuliah IF3270 Pembelajaran Mesin bertujuan agar mahasiswa mampu mengimplementasikan modul forward propagation untuk Convolutional Neural Network (CNN) serta Recurrent Neural Network (RNN/LSTM) secara mandiri dari awal (from scratch). Mahasiswa diwajibkan membangun berbagai jenis layer secara modular untuk CNN (Conv2D shared/non-shared, Pooling, Flatten, Dense) dan pemrosesan teks (Embedding, sel RNN/LSTM, Dense projection/output), beserta fungsi utilitas pemrosesan datanya.

Implementasi ini diaplikasikan pada dua kasus utama: image classification menggunakan dataset Intel Image untuk CNN, dan image captioning menggunakan dataset Flickr8k dengan arsitektur encoder-decoder berkonsep pre-inject. Pelatihan model dilakukan menggunakan pustaka Keras dengan loss Sparse Categorical Crossentropy, optimizer Adam, dan metode teacher forcing. Bobot dari model Keras tersebut kemudian dimuat (load) ke dalam modul from scratch untuk menjalankan proses inferensi.

Tahap akhir berfokus pada eksperimen dan evaluasi model. Kinerja CNN dianalisis menggunakan macro F1-score untuk mengevaluasi pengaruh modifikasi hyperparameter dan penggunaan shared vs non-shared parameters. Sementara itu, kinerja model image captioning dievaluasi menggunakan skor BLEU-4 dan METEOR terhadap variasi hyperparameter arsitektur RNN dan LSTM. Pengujian ini juga mencakup analisis kualitatif prediksi teks serta komparasi akurasi dan waktu eksekusi antara implementasi from scratch dengan bawaan Keras.

III. Pembahasan

A. Penjelasan Implementasi

1. CNN

Model CNN kami diimplementasikan dalam beberapa komponen layer yang dipisah di `src/cnn/layers.py`, sedangkan alur inferensi from-scratch-nya disusun di `src/cnn/model.py`. Semua layer from-scratch dibuat hanya dengan NumPy, lalu bobot dari model Keras dimuat kembali agar forward propagation dapat dijalankan secara mandiri. Untuk membantu melihat cara model membaca gambar, kami juga menambahkan visualisasi feature map dan Grad-CAM di `src/cnn/visualize.py` sebagai salah satu spesifikasi bonus.

a. Kelas Conv2DLayer

Lokasi: `src/cnn/layers.py`

Kelas `Conv2DLayer` merupakan implementasi convolutional layer dengan parameter sharing. Satu kernel yang sama dipakai berulang pada seluruh bagian input untuk membentuk feature map. Pada kode, bagian-bagian kecil input dibentuk menjadi patch, lalu dihitung dengan kernel untuk menghasilkan output konvolusi.

- Atribut

Atribut	Deskripsi
kernel	Bobot filter konvolusi bertipe float32, shape (kH, kW, C_in, C_out)
bias	Vektor bias, shape (C_out,)
stride_h, stride_w	Besar langkah perpindahan kernel pada arah tinggi dan lebar
padding	Jenis padding yang digunakan, yaitu "valid" atau "same"
kH, kW	Tinggi dan lebar kernel
_act_name, activation	Nama dan fungsi aktivasi yang digunakan
_cache	Menyimpan data penting dari forward pass untuk kebutuhan backward

- Method:

Method	Deskripsi
<code>__init__(kernel, bias, strides, padding, activation)</code>	Konstruktor, memuat bobot dan konfigurasi layer
<code>forward(x)</code>	Menjalankan konvolusi, menambahkan bias, lalu menerapkan aktivasi
<code>backward(dout)</code>	Menghitung gradien terhadap input, kernel, dan bias

b. Kelas `LocallyConnected2DLayer`

Lokasi: `src/cnn/layers.py`

Kelas `LocallyConnected2DLayer` mengimplementasikan convolutional layer tanpa parameter sharing. Berbeda dari `Conv2DLayer`, setiap posisi pada input memiliki kernel dan bias masing-masing. Dengan cara ini, model dapat belajar pola yang berbeda untuk posisi yang berbeda, tetapi jumlah parameternya menjadi jauh lebih banyak.

- Atribut

Atribut	Deskripsi
<code>kernel</code>	Bobot lokal untuk tiap posisi, shape (out_h, out_w, kH*kW*C_in, C_out)
<code>bias</code>	Bias lokal untuk tiap posisi, shape (out_h, out_w, C_out)
<code>kH, kW</code>	Tinggi dan lebar kernel
<code>stride_h, stride_w</code>	Besar langkah perpindahan kernel
<code>padding</code>	Jenis padding yang digunakan
<code>_act_name, activation</code>	Nama dan fungsi aktivasi
<code>_cache</code>	Menyimpan data forward pass untuk backward

- Method:

Method	Deskripsi
<code>__init__(kernel, bias, kernel_size, strides, padding, activation)</code>	Konstruktor, memuat bobot lokal dan konfigurasi layer
<code>forward(x)</code>	Menghitung output pada setiap posisi menggunakan kernel lokal masing-masing
<code>backward(dout)</code>	Menghitung gradien terhadap input, kernel lokal, dan bias lokal

c. Kelas Pooling

Lokasi: `src/cnn/layers.py`

Pooling diimplementasikan menjadi empat kelas, yaitu `MaxPooling2DLayer`, `AvgPooling2DLayer`, `GlobalMaxPooling2DLayer`, dan `GlobalAvgPooling2DLayer`. Pooling biasa dipakai untuk mengecilkan ukuran feature map, sedangkan pooling global merangkum seluruh area spasial menjadi satu nilai untuk setiap channel.

- Kelas

Kelas	Fungsi Utama
<code>MaxPooling2DLayer</code>	Mengambil nilai maksimum pada setiap window lokal
<code>AvgPooling2DLayer</code>	Mengambil nilai rata-rata pada setiap window lokal
<code>GlobalMaxPooling2DLayer</code>	Mengambil nilai maksimum dari seluruh area spasial tiap channel
<code>GlobalAvgPooling2DLayer</code>	Mengambil nilai rata-rata dari seluruh area spasial tiap channel

- Atribut Umum:

Atribut	Deskripsi
<code>pH, pW</code>	Ukuran window pooling pada pooling non-global
<code>stride_h, stride_w</code>	Langkah perpindahan window pooling

<code>_cache</code>	Menyimpan informasi input atau shape dari forward pass
---------------------	--

- Method:

Method	Deskripsi
<code>forward(x)</code>	Menghasilkan representasi hasil pooling
<code>backward(dout)</code>	Mengembalikan gradien ke ukuran input sebelum pooling

d. Kelas FlattenLayer

Lokasi: `src/cnn/layers.py`

Kelas FlattenLayer mengubah feature map yang masih memiliki ukuran tinggi dan lebar menjadi vektor satu dimensi agar bisa diproses oleh dense layer.

- Atribut

Atribut	Deskripsi
<code>_cache</code>	Menyimpan shape input sebelum proses flatten

- Method:

Method	Deskripsi
<code>forward(x)</code>	Mengubah tensor menjadi bentuk 1D untuk satu sampel atau 2D untuk batch
<code>backward(dout)</code>	Mengembalikan tensor ke shape sebelum flatten

e. Kelas DenseLayer

Lokasi: `src/cnn/layers.py`

Kelas DenseLayer merupakan implementasi fully-connected layer. Layer ini dipakai untuk mengubah hasil ekstraksi fitur menjadi skor untuk setiap kelas.

- Atribut

Atribut	Deskripsi
---------	-----------

kernel	Matrix bobot, shape (input_dim, output_dim)
bias	Vektor bias, shape (output_dim,)
_act_name, activation	Nama dan fungsi aktivasi
_cache	Menyimpan input, nilai sebelum aktivasi, dan output

- Method:

Method	Deskripsi
__init__(kernel, bias, activation)	Konstruktor, memuat bobot dense dari Keras
forward(x)	Menghitung $z = xW + b$, lalu menerapkan aktivasi jika ada
backward(dout)	Menghitung gradien terhadap input, kernel, dan bias

f. Kelas BatchNormLayer

Lokasi: src/cnn/layers.py

Kelas BatchNormLayer mengimplementasikan batch normalization untuk kebutuhan inferensi. Parameter yang dipakai diambil dari model Keras hasil training, yaitu moving mean, moving variance, gamma, dan beta.

- Atribut

Atribut	Deskripsi
gamma	Parameter skala
beta	Parameter geser
mean	Moving mean dari proses training
var	Moving variance dari proses training
eps	Nilai kecil untuk menjaga kestabilan pembagian
_cache	Menyimpan nilai hasil normalisasi untuk backward

- Method:

Method	Deskripsi
<code>__init__(gamma, beta, moving_mean, moving_var, epsilon)</code>	Konstruktor, memuat parameter batch normalization
<code>forward(x)</code>	Menjalankan normalisasi pada input
<code>backward(dout)</code>	Menghitung gradien terhadap input, gamma, dan beta

g. Kelas `CNNFromScratch`

Lokasi: `src/cnn/model.py`

Kelas `CNNFromScratch` berfungsi sebagai penghubung antara model Keras dan implementasi NumPy. Kelas ini membaca layer-layer aktif dari model Keras, mengubahnya ke layer from-scratch yang sesuai, lalu menjalankan forward propagation secara berurutan.

- Atribut

Atribut	Deskripsi
<code>layers</code>	Daftar layer aktif hasil konversi dari model Keras

- Method:

Method	Deskripsi
<code>from_keras(keras_model)</code>	Class method untuk mengubah model Keras menjadi rangkaian layer from-scratch
<code>predict(x, batch_size)</code>	Menjalankan forward propagation untuk satu input atau batch input
<code>predict_classes(X, batch_size)</code>	Menghasilkan kelas akhir dengan mengambil indeks nilai output terbesar

Pada proses konversi, beberapa layer yang tidak dibutuhkan saat inferensi, seperti `InputLayer`, `Dropout`, `Rescaling`, dan `augmentasi data`, dilewati. Layer utama seperti `Conv2D`, `LocallyConnected2D`, `pooling`, `Flatten`, `Dense`, dan `BatchNormalization` kemudian dipetakan ke kelas from-scratch yang sesuai.

h. Utilitas Visualisasi (Bonus)

Lokasi: `src/cnn/visualize.py`

Selain pipeline utama, modul CNN juga memiliki beberapa fungsi visualisasi untuk membantu melihat hasil kerja model:

- Fungsi

Fungsi	Deskripsi
<code>plot_feature_maps(...)</code>	Menampilkan feature map dari setiap convolutional layer pada model from-scratch
<code>grad_cam(...)</code>	Menghitung heatmap Grad-CAM dari model Keras
<code>plot_grad_cam(...)</code>	Menampilkan citra asli, heatmap, dan overlay Grad-CAM
<code>plot_grad_cam_grid(...)</code>	Menampilkan beberapa hasil Grad-CAM dalam satu grid

2. Simple RNN & LSTM

Model Simple RNN dan LSTM dibangun menggunakan kelas utama `CaptioningFromScratch` yang merupakan implementasi manual dari arsitektur image captioning dengan berbagai kelas pendukung seperti `EmbeddingLayer`, `SimpleRNNCell`, `LSTMCell`, dan `DenseLayer`.

a. Kelas `EmbeddingLayer`

Lokasi: `src/rnn_lstm/layers.py`

Kelas `EmbeddingLayer` adalah layer yang berfungsi untuk mengkonversi token ID menjadi representasi vektor dense (embedding) dengan dimensi tertentu.

- Atribut:

Atribut	Deskripsi
<code>W</code>	Matrix bobot embedding dengan shape (vocab_size, embed_dim)
<code>embed_dim</code>	Dimensi vektor embedding, diambil dari <code>W.shape[1]</code>
<code>_cache_token</code>	Menyimpan token ID terakhir untuk keperluan

	backward
_cache_tokens	Menyimpan array token ID terakhir untuk keperluan backward pada batch

- Method:

Method	Deskripsi
__init__(embedding_matrix)	Konstruktor untuk inisialisasi layer dengan matrix bobot embedding
forward(token_id)	Melakukan forward pass untuk satu token dan mengembalikan vektor embedding dengan shape (embed_dim,)
forward_batch(token_ids)	Melakukan forward pass untuk batch token dan mengembalikan matrix dengan shape (N, embed_dim)
forward_sequence(token_ids)	Melakukan forward pass untuk seluruh sequence sekaligus
backward(dout, token_ids=None)	Melakukan backward pass untuk menghitung gradien terhadap bobot embedding

b. Kelas SimpleRNNCell

Lokasi: src/rnn_lstm/layers.py

Kelas SimpleRNNCell adalah implementasi sel RNN dasar yang memproses satu timestep dan memperbarui hidden state dengan persamaan $h_{\text{new}} = \tanh(x \cdot W_x + h \cdot W_h + b)$.

- Atribut:

Atribut	Deskripsi
W_x	Matrix bobot untuk input ke hidden state
W_h	Matrix bobot untuk koneksi recurrent (hidden state sebelumnya ke hidden state baru)
b	Vektor bias
units (property)	Jumlah unit dalam sel RNN

<code>_cache</code>	Dictionary menyimpan <code>x</code> , <code>h</code> , <code>h_new</code> dari forward pass terakhir untuk backward
---------------------	---

- Method:

Method	Deskripsi
<code>__init__(kernel, recurrent_kernel, bias)</code>	Konstruktor untuk inisialisasi sel dengan bobot dan bias dari Keras
<code>forward(x, h)</code>	Melakukan forward pass untuk satu timestep dan mengembalikan hidden state baru
<code>backward(dh_new)</code>	Forward pass untuk batch token, mengembalikan matrix shape $(N, \text{embed_dim})$
<code>forward_sequence(token_ids)</code>	Melakukan backward pass BPTT dan mengembalikan gradien $(dx, dh_prev, dW_x, dW_h, db)$
<code>zero_state(batch_size=1)</code>	Mengembalikan hidden state awal berupa nol dengan shape $(\text{batch_size}, \text{units})$

c. Kelas LSTMCell

Lokasi: `src/rnn_lstm/layers.py`

Kelas `LSTMCell` adalah implementasi sel LSTM yang memiliki empat gate yaitu input gate, forget gate, cell gate, dan output gate untuk mengatasi masalah vanishing gradient pada sequence yang panjang.

- Atribut:

Atribut	Deskripsi
<code>W_x</code>	Matrix bobot untuk input ke semua gate dengan shape $(\text{input_dim}, 4 \times \text{units})$
<code>W_h</code>	Matrix bobot recurrent ke semua gate dengan shape $(\text{units}, 4 \times \text{units})$
<code>b</code>	Vektor bias gabungan untuk 4 gate dengan shape $(4 \times \text{units},)$
<code>units</code>	Jumlah unit dalam sel LSTM
<code>_cache</code>	Menyimpan nilai <code>x</code> , <code>h</code> , <code>c</code> , gate <code>i/f/g/o</code> , <code>c_new</code> , dan

	h_new untuk keperluan backward
--	--------------------------------

- Method:

Method	Deskripsi
<code>__init__(kernel, recurrent_kernel, bias)</code>	Konstruktor untuk inisialisasi sel dengan bobot dan bias dari Keras
<code>forward(x, h, c)</code>	Melakukan forward pass untuk satu timestep dengan menghitung 4 gate dan menghasilkan h_new dan c_new
<code>backward(dh_new, dc_new)</code>	Melakukan backward pass BPTT dan mengembalikan gradien (dx, dh_prev, dc_prev, dW_x, dW_h, db)
<code>zero_state(batch_size=1)</code>	Mengembalikan pasangan (h, c) nol dengan shape (batch_size, units)

d. Kelas DenseLayer

Lokasi: `src/rnn_lstm/layers.py`

Kelas DenseLayer adalah implementasi fully-connected layer yang digunakan untuk dua tujuan: proyeksi fitur gambar ke dimensi hidden (projection dense) dan menghasilkan distribusi probabilitas atas vocabulary (output dense dengan softmax).

- Atribut:

Atribut	Deskripsi
<code>W</code>	Matrix bobot dengan shape (input_dim, output_dim)
<code>b</code>	Vektor bias dengan shape (output_dim,)
<code>activation</code>	Nama fungsi aktivasi berupa "softmax", "relu", atau None untuk linear
<code>_cache</code>	Menyimpan nilai x, z, dan out untuk keperluan backward

- Method:

Method	Deskripsi
--------	-----------

<code>__init__(kernel, bias, activation=None)</code>	Konstruktor untuk inisialisasi layer dengan bobot dan bias dari Keras
<code>forward(x)</code>	Melakukan forward pass dengan menghitung $z = x \cdot W + b$ dan mengaplikasikan fungsi aktivasi
<code>backward(dout)</code>	Melakukan backward pass dan mengembalikan gradien (dx, dW, db)

e. Kelas CaptioningFromScratch

Lokasi: `src/rnn_lstm/model.py`

Kelas `CaptioningFromScratch` adalah kelas utama yang mengintegrasikan CNN encoder dengan decoder from-scratch dan mendukung dua arsitektur injection yaitu pre-inject dan init-inject untuk kedua jenis decoder RNN maupun LSTM.

- Atribut:

Atribut	Deskripsi
<code>cnn_encoder</code>	Keras model CNN untuk ekstraksi fitur gambar
<code>embedding</code>	Instance dari <code>EmbeddingLayer</code>
<code>proj_dense</code>	Instance dari <code>DenseLayer</code> untuk proyeksi fitur CNN ke dimensi hidden
<code>rnn_cells</code>	List berisi satu atau lebih <code>SimpleRNNCell</code> atau <code>LSTMCell</code>
<code>output_dense</code>	Instance dari <code>DenseLayer</code> dengan softmax untuk distribusi vocabulary
<code>arch</code>	Arsitektur injection yang digunakan yaitu 'inject' atau 'init_inject'
<code>img_size</code>	Ukuran gambar input untuk CNN encoder
<code>preprocess_fn</code>	Fungsi preprocessing gambar
<code>is_lstm</code>	Boolean yang bernilai True jika decoder menggunakan <code>LSTMCell</code>

- Method:

Method	Deskripsi
--------	-----------

<code>__init__(...)</code>	Konstruktor untuk inisialisasi semua komponen pipeline
<code>from_keras(cls, ...)</code>	Class method untuk memuat bobot dari Keras decoder dan mendeteksi arsitektur inject secara otomatis
<code>_extract_feature(img_path)</code>	Melakukan ekstraksi feature vector dari gambar menggunakan CNN encoder
<code>_init_states(batch_size)</code>	Menginisialisasi hidden state dan cell state dengan nilai nol
<code>_step(x, states)</code>	Melakukan satu langkah forward melalui seluruh stack RNN atau LSTM
<code>_decode(...)</code>	Melakukan greedy decode untuk arsitektur pre-inject
<code>_decode_init_inject(...)</code>	Melakukan greedy decode untuk arsitektur init-inject
<code>_decode_batch(...)</code>	Melakukan batch greedy decode untuk pre-inject secara paralel
<code>_decode_batch_init_inject(...)</code>	Melakukan batch greedy decode untuk init-inject secara paralel
<code>_beam_search_single(..)</code>	Melakukan beam search decode dengan length normalization
<code>generate_caption(img_path, ...)</code>	Interface publik untuk menghasilkan caption dari path gambar
<code>generate_from_feature(cnn_feature, ...)</code>	Interface publik untuk menghasilkan caption dari feature vector dengan greedy decode
<code>generate_from_feature_beam(cnn_feature, ...)</code>	Interface publik untuk menghasilkan caption dari feature vector dengan beam search
<code>generate_batch(cnn_features, ...)</code>	Interface publik untuk batch inference greedy pada N gambar
<code>generate_batch_beam(cnn_features, ...)</code>	Interface publik untuk batch inference beam search pada N gambar

B. Penjelasan Forward Propagation

1. CNN

Implementasi dari *forward propagation* dari CNN kami tidak jauh berbeda dengan apa yang telah dijelaskan di *slides* perkuliahan. Untuk memberi gambaran, kita dapat melihat CNN menjadi lima (5) bagian/*layer* berbeda dengan fungsinya masing-masing.

a. Convolutional Layer [Conv2DLayer / LocallyConnected2DLayer]

Pada layer ini, Input akan melewati sebuah *kernel* dengan tujuan melakukan ekstraksi fitur spasial dari *input* terkait. Sesuai dengan apa yang diajarkan, setiap bagian dari *input* akan melewati *kernel* setelah melakukan konfigurasi *padding* terhadap ukuran *input* dan *stride* terhadap *kernel*. Hasil dari *convolutional layer* adalah sebuah *matrix* hasil operasi konvolusi dengan menggunakan *element-wise multiplication* antara *input* dan *kernel*.

Kami mengimplementasikan dua jenis *convolutional layer*, yaitu Conv2DLayer dan LocallyConnected2DLayer. Pada umumnya, CNN akan menggunakan satu buah *kernel* yang digunakan oleh setiap bagian-bagian kecil dari *input* atau yang dinamakan *parameter sharing*. Itu apa yang diimplementasi pada Conv2DLayer. Untuk LocallyConnected2DLayer, kami mengimplementasi untuk setiap bagian-bagian kecil dari *input* memiliki *kernel*-nya masing-masing yang berbeda satu sama yang lain (meskipun bisa sama untuk beberapa).

Pada setiap jenisnya, operasi konvolusi juga melibatkan bobot dari *bias* yang terpisah dari *kernel* yang akan ditambahkan ke hasil akhir dari *element-wise multiplication* yang sudah dijelaskan. Setelah itu semua, dilakukan juga *activation*. *Activation* dapat menggunakan berbagai cara, seperti ReLU, *softmax*, *sigmoid*, *tanh*, dan *linear*.

ReLU	Softmax	Sigmoid	Tanh	Linear
$\text{MAX}(0, x)$	$\frac{e^{x_i}}{\sum_{n=1}^N e^{x_n}}$	$\frac{1}{1+e^{-x}}$	$\frac{e^{2x}-1}{e^{2x}+1}$	x

b. Batch Normalization

Batch Normalization adalah teknik normalisasi yang diterapkan pada output dari sebuah layer sebelum diteruskan ke layer berikutnya. Tujuannya adalah untuk menstabilkan distribusi nilai aktivasi selama proses pembelajaran sehingga model dapat belajar lebih cepat dan lebih stabil. Secara sederhana, layer ini akan menormalisasi nilai output dengan mengurangi rata-rata batch dan membaginya dengan standar deviasi batch, kemudian mengalikannya dengan parameter gamma dan menambahkan parameter beta yang dapat dipelajari oleh model.

c. Pooling Layer

Layer ini memiliki fungsi utama untuk melakukan reduksi ukuran spasial dari sebuah *input*. Hal tersebut dilakukan dengan alasan mencegah *overfitting* dalam pembelajaran dan mendukung generalisasi yang baik untuk *model* yang dimaksud. Layer ini menggunakan empat (4) variasi *pooling* dengan dua (2) jenis kalkulasi yang dilakukan. Max *pooling* adalah jenis kalkulasi yang mengambil satu nilai, yaitu yang terbesar, dari setiap *sub-patch*. Average *pooling* adalah jenis kalkulasi yang menghitung rata-rata semua nilai untuk dijadikannya *value output* dari *sub-patch* tersebut. Perbedaan dari variasi *global* dan tidak *global* adalah tujuannya. Untuk *global*, hasil dari *pooling* akan hanya 1 karena akan di-*collapse* spasialnya. Untuk *non-global*, akan mempertahankan spasial dengan ukuran yang lebih kecil.

d. Flatten Layer

Layer ini untuk menghapus spasial dari *matrix* dengan cara melakukan *flatten* menjadi bentuk 1 dimensi saja. Hal tersebut untuk memungkinkan *model* melakukan pembelajaran dan *inference* kedepannya.

e. Dense Layer

Layer ini digunakan untuk menghubungkan setiap *neuron input* ke setiap *neuron output* untuk mempertahankan konsep *fully connected* pada Convolution Neural Network. Sederhananya, ini untuk dapat menyesuaikan dengan jumlah kategori yang harus dipilih nantinya oleh *model*.

2. Simple RNN & LSTM

Implementasi dari RNN & LSTM kami sebenarnya tidak sekedar RNN & LSTM saja. Konsep yang kami implementasi seperti apa yang dijelaskan mengenai *encoder* & *decoder* dalam perkuliahan. Untuk RNN & LSTM lebih fokus pada bagian *decoder* tentunya dengan *encoder* didukung dengan CNN. Untuk mempermudah penjelasan terkait RNN & LSTM, kita dapat pecah menjadi delapan (8) bagian, yaitu:

a. CNN Layer

Disini kita akan menerima *input* berupa gambar yang dilakukan *resize* untuk nantinya dimasukkan ke sebuah CNN, dimana pada kasus ini kami menggunakan InceptionV3 (*pre-trained*). CNN nantinya, seperti yang sudah dijelaskan sebelum, akan menghasilkan vektor *feature* sebagai representasi konten dari *input gambar* dalam bentuk numerik yang dapat diproses dan dipelajari oleh *model*.

b. Dense Layer

Pada kenyataannya, hasil dari CNN umumnya terlalu besar untuk keperluan RNN & LSTM melakukan pembelajaran dengan efisien. Sehingga, kami mengimplementasi layer ini untuk melakukan reduksi ukuran vektor dengan mempertahankan konektivitas dengan menggunakan *dense layer*.

c. Inject Feature

Inject Feature merupakan mekanisme yang menghubungkan informasi visual dari encoder CNN ke dalam proses pembangkitan kata pada decoder. Terdapat dua cara injeksi yang berbeda yaitu pre-inject dan init-inject. Pada pre-inject, vektor fitur gambar digunakan sekali di awal untuk mengatur kondisi awal hidden state sebelum pembangkitan kata pertama dimulai, sehingga informasi gambar menjadi titik awal dari seluruh proses decoding.

Pada init-inject, vektor fitur gambar tidak hanya berperan di awal saja melainkan ditambahkan ke hidden state pada setiap langkah waktu selama proses pembangkitan caption berlangsung. Pendekatan ini memastikan bahwa setiap kata yang dihasilkan selalu mempertimbangkan konteks visual gambar secara langsung, bukan hanya mengandalkan informasi yang tersimpan dari awal.

Pada arsitektur init-inject, vektor fitur tidak hanya diinjeksikan di awal saja melainkan ditambahkan ke hidden state di setiap langkah waktu pembangkitan kata. Hal ini memungkinkan model untuk selalu mempertimbangkan informasi visual dari gambar pada setiap kata yang dihasilkan sehingga konteks gambar tidak hilang meskipun caption yang dihasilkan semakin panjang.

d. Embedding

Untuk *model* dapat melakukan pembelajaran, kata harus diubah menjadi bentuk angka. Secara umum, *caption* ataupun kata lainnya akan dilakukan *embedding* menjadi sebuah *matrix*. Pada implementasinya, sebuah kata akan diubah menjadi sebuah *integer* yang sebenarnya hanya `token_id` sebagai index dari *matrix* kata tersebut.

e. RNN/LSTM Forward

Bagian ini menjadi implementasi apa yang sudah diajarkan mengenai RNN & LSTM saat di kelas. Untuk RNN, sekedar menggunakan perhitungan $z = (W_{xh} \cdot x + W_{hh} \cdot h + b)$ lalu melakukan aktivasi dari hasil z menggunakan $\tanh(z)$. Untuk LSTM akan lebih panjang dibandingkan RNN karena menggunakan 4 *gate* tambahan. Namun, secara ringkas akan menghitung dua (2) hal, yaitu Long Term Memory ($c^{(t)}$) dan Short Term Memory ($h^{(t)}$).

f. Prediction

Sebenarnya, terdapat dua (2) strategi yang diimplementasi untuk melakukan *prediction*. Pertama, Greedy Decoding yang dimana *output* dari *hidden state* dari RNN/LSTM dimasukkan ke *output dense* dengan *activation function* softmax. Token nanti akan dipilih dengan argmax yang dimana akan menjadi *input embedding* untuk langkah-langkah berikutnya. Itu akan terus berjalan hingga sebuah *token <end>* diprediksi atau `max_len` sudah tercapai.

Kedua, Beam Search yang dimana akan mempertahankan k banyaknya kandidat untuk *caption* (*greedy* hanya memilih 1 kata terbaik tanpa memikirkan keseluruhan kalimat). Di setiap langkah, kandidat dicoba untuk dilanjutkan dengan semua kemungkinan kata berikutnya, lalu akan hanya mempertahankan k banyaknya kombinasi yang terbaik. Proses mengkombinasi ($k \times k$) kandidat dan

mempertahankan k yang terbaik akan terus berlanjut hingga sebuah *token* <end> diprediksi atau `max_len` sudah tercapai.

C. Hasil Pengujian

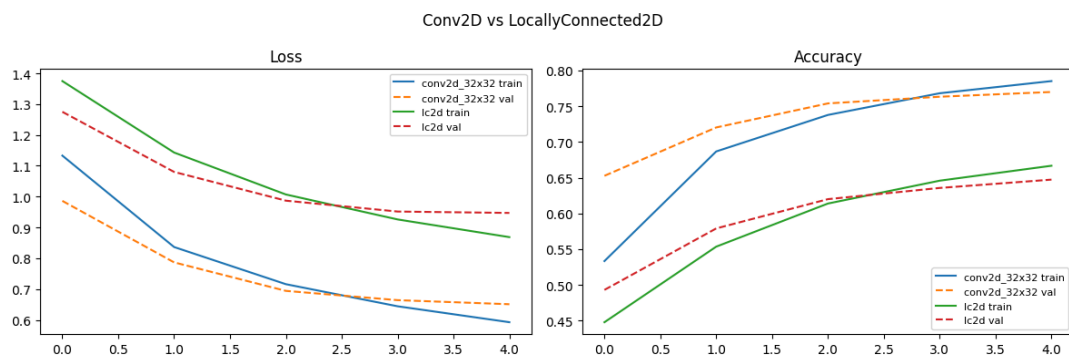
1. CNN

Sebagai catatan, kami mengimplementasikan *model* dalam CNN dengan adanya *early stopping* dengan pengaturan *patience* di angka lima (5). Hal tersebut membuat beberapa grafik terpotong akibat dari sudah mulai *converge* di masing-masing variasi.

A) Perbandingan shared vs non-shared parameter

Tipe Layer	F1-macro	Jumlah Parameter
Conv2D (Parameter Sharing)	0.6485	65,446
LC2D (Non-Shared)	0.4451	10,397,830

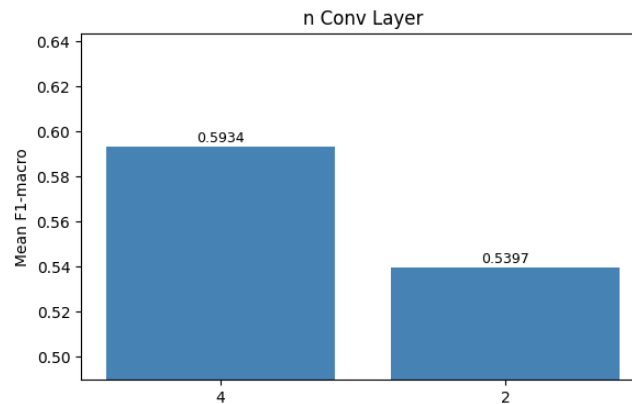
Berdasarkan tabel perbandingan di atas, Conv2D dengan parameter sharing menghasilkan F1-macro sebesar 0.6485 dengan hanya 65.446 parameter, sedangkan LC2D tanpa parameter sharing hanya mencapai 0.4451 meskipun memiliki 10.397.830 parameter atau sekitar 159 kali lebih banyak. Hal ini menunjukkan bahwa parameter sharing pada Conv2D bukan hanya menghemat jumlah parameter, tetapi juga berperan sebagai regularisasi implisit yang memaksa model mempelajari fitur yang lebih general dan tidak bergantung pada posisi spasial, sehingga menghasilkan generalisasi yang lebih baik pada data pengujian.



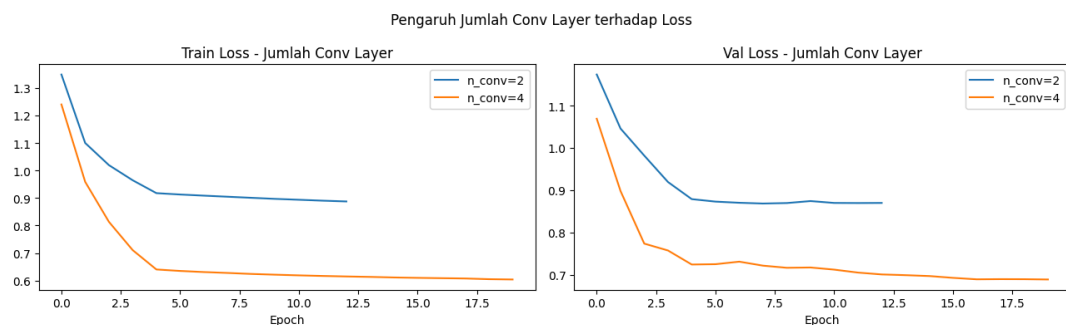
Dari grafik training dan validation loss terlihat bahwa Conv2D menunjukkan konvergensi yang lebih mulus dengan gap kecil antara training loss dan validation loss. Sebaliknya, LC2D memiliki validation loss

yang jauh lebih tinggi dibandingkan training loss sepanjang proses pelatihan, yang merupakan indikasi overfitting akibat kapasitas model yang jauh melebihi kebutuhan dataset.

B) Pengaruh jumlah layer konvolusi



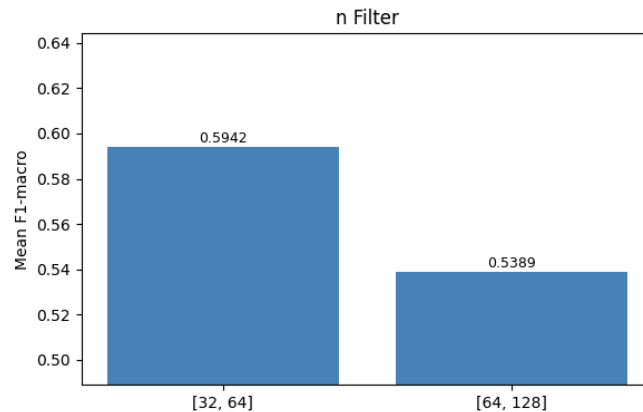
Berdasarkan grafik rata-rata F1-score per variasi jumlah layer konvolusi, model dengan $n_{\text{conv}}=4$ menghasilkan rata-rata F1-macro sebesar 0.5934, sedangkan model dengan $n_{\text{conv}}=2$ hanya mencapai rata-rata 0.5397. Perbedaan ini semakin terlihat jelas pada konfigurasi terbaik yaitu $n_{\text{conv}}=4$ dengan filter [32, 64], kernel size 3, dan max pooling yang berhasil mencapai F1-macro tertinggi sebesar 0.8536. Sehingga dapat disimpulkan bahwa jumlah layer konvolusi yang lebih banyak memberikan kapasitas representasi yang lebih besar dan menghasilkan performa yang lebih baik pada dataset ini.



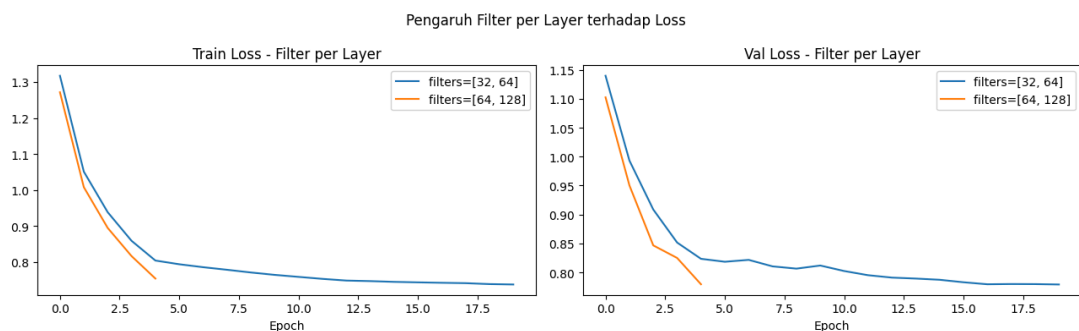
Jika melihat grafik training dan validation loss untuk variasi jumlah layer konvolusi, model dengan $n_{\text{conv}}=4$ memerlukan lebih banyak epoch untuk konvergen namun mencapai loss akhir yang lebih rendah, sementara model dengan $n_{\text{conv}}=2$ cenderung berhenti lebih cepat akibat early stopping dengan loss yang lebih tinggi. Tidak ada tanda-tanda overfitting yang signifikan pada kedua variasi, hal ini

kemungkinan dikarenakan rentang nilai `n_conv` yang digunakan relatif kecil yaitu hanya 2 dan 4 sehingga perbedaan kapasitas model tidak cukup besar untuk menyebabkan overfitting.

C) Pengaruh banyak filter per layer konvolusi



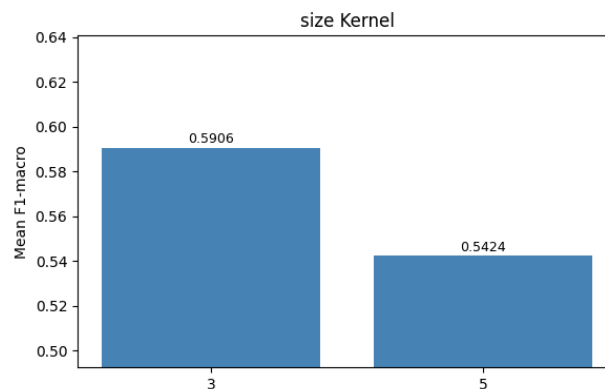
Berdasarkan grafik rata-rata F1-score per variasi jumlah filter, model dengan filter [32, 64] menghasilkan rata-rata F1-macro sebesar 0.5942, sedangkan model dengan filter [64, 128] hanya mencapai rata-rata 0.5389. Dua konfigurasi teratas juga keduanya menggunakan filter [32, 64] yaitu `c4_f32_64_k3_max` dengan F1 0.8536 dan `c2_f32_64_k3_max` dengan F1 0.7041. Hal ini cukup menarik karena secara intuitif filter yang lebih besar seharusnya mengekstraksi lebih banyak fitur, namun pada dataset ini filter yang lebih kecil justru menghasilkan performa yang lebih baik. Kemungkinan besar hal ini dikarenakan filter yang lebih besar menghasilkan jumlah parameter yang jauh lebih banyak sehingga model lebih rentan terhadap overfitting pada dataset yang relatif terbatas.



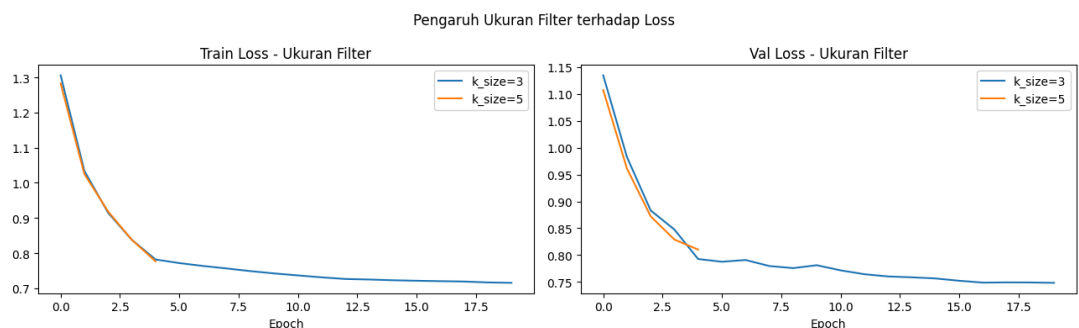
Jika melihat grafik training dan validation loss untuk variasi jumlah filter, model dengan filter [32, 64] memerlukan lebih banyak epoch untuk konvergen yaitu hingga sekitar epoch 19 dengan loss yang terus menurun secara bertahap. Sebaliknya, model dengan filter [64, 128] berhenti lebih

awal sekitar epoch 4 akibat early stopping dengan nilai loss yang sedikit lebih tinggi. Hal ini menunjukkan bahwa model dengan filter yang lebih besar memang memiliki kecepatan konvergensi yang lebih cepat di awal, namun tidak mampu terus meningkatkan performanya sehingga terhenti lebih awal, sedangkan model dengan filter yang lebih kecil memiliki ruang yang lebih besar untuk terus belajar dan menghasilkan loss akhir yang lebih rendah.

D) Pengaruh ukuran filter per layer konvolusi



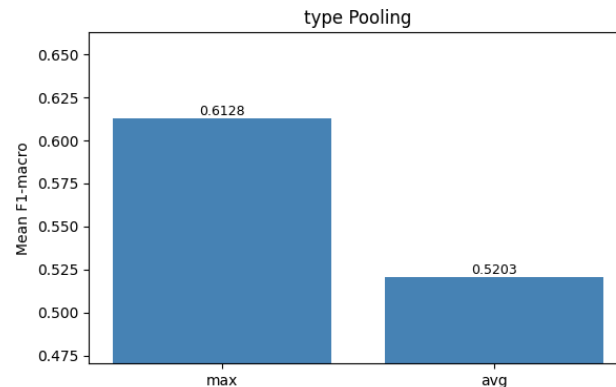
Berdasarkan grafik rata-rata F1-score per variasi ukuran filter, model dengan kernel size 3 menghasilkan rata-rata F1-macro sebesar 0.5906, sedangkan model dengan kernel size 5 hanya mencapai rata-rata 0.5424. Kernel yang lebih kecil berukuran 3x3 mampu menangkap pola lokal dengan lebih presisi dan efisien pada dataset ini, sementara kernel yang lebih besar berukuran 5x5 menghasilkan jumlah parameter yang lebih banyak tanpa memberikan peningkatan performa yang signifikan.



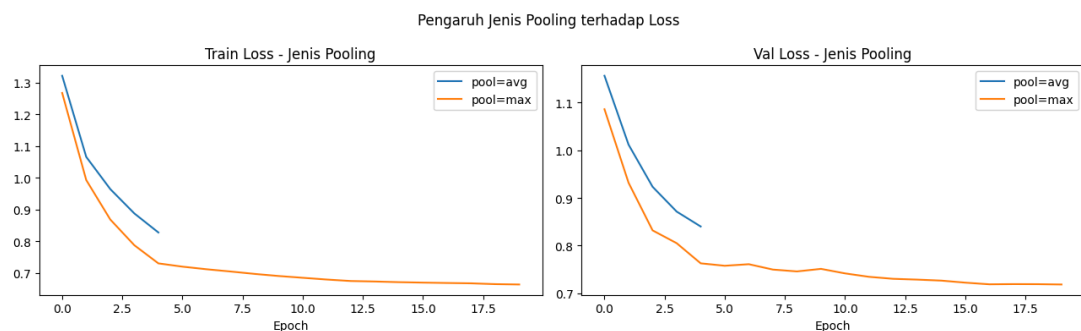
Jika melihat grafik training dan validation loss untuk variasi ukuran filter, kedua kurva menunjukkan pola yang sangat mirip di awal pelatihan dengan nilai loss yang hampir berhimpit. Perbedaan mulai terlihat setelah beberapa epoch di mana model dengan kernel size 3 terus melanjutkan pelatihan hingga sekitar epoch 19 dan berhasil mencapai train loss sekitar

0.71 serta val loss sekitar 0.75, sedangkan model dengan kernel size 5 berhenti lebih awal sekitar epoch 4 akibat early stopping dengan loss akhir yang lebih tinggi. Hal ini menunjukkan bahwa model dengan kernel size 3 memiliki kapasitas yang lebih sesuai dengan kompleksitas dataset sehingga masih dapat terus meningkat, sedangkan model dengan kernel size 5 mencapai batas konvergensinya lebih cepat.

E) Pengaruh jenis pooling layer



Berdasarkan grafik rata-rata F1-score per variasi jenis pooling, model dengan max pooling menghasilkan rata-rata F1-macro sebesar 0.6128, sedangkan model dengan average pooling hanya mencapai rata-rata 0.5203. Perbedaan ini merupakan yang paling besar dibandingkan variasi hyperparameter lainnya, menunjukkan bahwa jenis pooling memiliki pengaruh yang cukup signifikan terhadap performa model. Max pooling yang mengambil nilai maksimum dari setiap window lebih efektif dalam mempertahankan fitur-fitur dominan seperti tepi dan tekstur yang penting untuk membedakan kelas pada dataset Intel Image Classification.



Jika melihat grafik training dan validation loss untuk variasi jenis pooling, model dengan max pooling menunjukkan konvergensi yang jauh

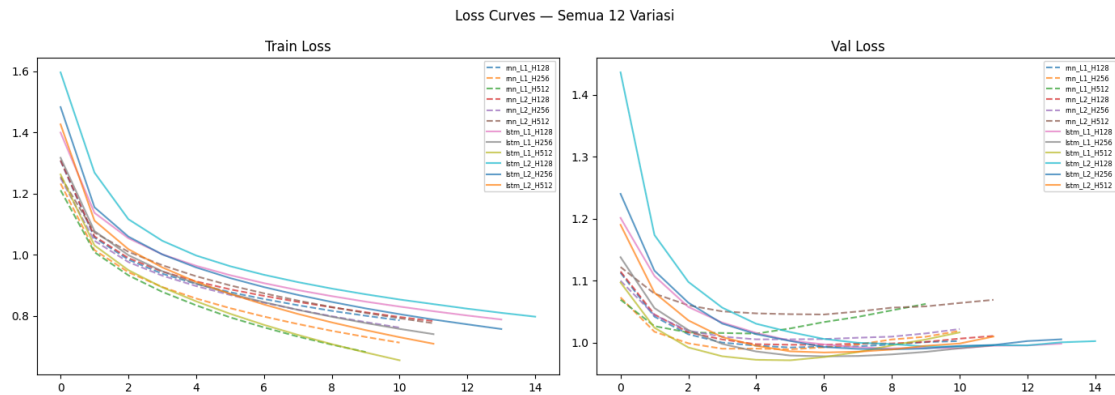
lebih cepat dan mencapai loss yang lebih rendah dibandingkan average pooling (biru). Model dengan max pooling terus melanjutkan pelatihan hingga sekitar epoch 19 dengan train loss sekitar 0.68 dan val loss sekitar 0.70, sedangkan model dengan average pooling berhenti lebih awal dengan loss akhir yang lebih tinggi. Hal ini menunjukkan bahwa max pooling tidak hanya menghasilkan performa akhir yang lebih baik tetapi juga memiliki dinamika pelatihan yang lebih stabil dan efisien pada dataset ini.

F) Perbandingan Keras vs from scratch

Implementasi	F1-macro	Waktu (s)
Keras	0.8536	1.40
From-Scratch	0.8536	41.15
Prediksi Identik: True		

Berdasarkan hasil pengujian pada data test, implementasi forward propagation from scratch menghasilkan F1-macro sebesar 0.8536 yang identik dengan hasil dari Keras. Hal ini juga dikonfirmasi oleh nilai "Prediksi identik: True" yang menunjukkan bahwa setiap prediksi per sampel pada kedua implementasi menghasilkan kelas yang sama persis. Keidentikan hasil ini membuktikan bahwa implementasi from scratch telah berhasil mereplikasi operasi konvolusi, pooling, dan fungsi aktivasi secara akurat sesuai dengan bobot yang dilatih menggunakan Keras. Perbedaan yang terlihat hanya pada waktu inferensi di mana Keras membutuhkan 1.40 detik sedangkan from scratch membutuhkan 41.15 detik, hal ini wajar karena Keras memanfaatkan backend yang dioptimasi dengan akselerasi GPU sedangkan implementasi from scratch menggunakan operasi NumPy murni tanpa optimasi serupa.

2. Simple RNN dan LSTM untuk Image Captioning



	model	cell	n_layers	hidden	bleu4	meteor	time
0	lstm_L2_H512	lstm	2	512	0.0914	0.2465	72.27
1	rnn_L1_H512	rnn	1	512	0.0794	0.2280	17.96
2	lstm_L2_H256	lstm	2	256	0.0737	0.2414	25.62
3	lstm_L1_H512	lstm	1	512	0.0685	0.2293	39.28
4	rnn_L2_H512	rnn	2	512	0.0676	0.2106	22.04
5	lstm_L2_H128	lstm	2	128	0.0675	0.2664	23.51
6	rnn_L1_H128	rnn	1	128	0.0649	0.2266	18.35
7	lstm_L1_H256	lstm	1	256	0.0600	0.2289	24.15
8	rnn_L2_H256	rnn	2	256	0.0590	0.2312	15.44
9	rnn_L1_H256	rnn	1	256	0.0574	0.2040	15.03
10	lstm_L1_H128	lstm	1	128	0.0559	0.2340	14.98
11	rnn_L2_H128	rnn	2	128	0.0530	0.2225	10.42

A) Perbandingan jumlah layer: RNN

Berdasarkan tabel hasil evaluasi, model RNN dengan 1 layer menghasilkan rata-rata BLEU-4 sebesar 0.0672, sedangkan model RNN dengan 2 layer hanya mencapai rata-rata 0.0599. Jika melihat grafik training dan validation loss, kurva RNN 1 layer cenderung memiliki loss akhir yang lebih rendah dibandingkan 2 layer. Hal ini kemungkinan disebabkan oleh penambahan layer kedua yang memperparah

vanishing gradient pada RNN sehingga model kesulitan mempelajari dependensi jangka panjang yang dibutuhkan untuk menghasilkan caption yang koheren.

B) Perbandingan jumlah layer: LSTM

Berbeda dengan RNN, model LSTM dengan 2 layer justru mengungguli 1 layer dengan rata-rata BLEU-4 sebesar 0.0775 dibandingkan 0.0615. Dari grafik loss, LSTM 2 layer pada beberapa konfigurasi menunjukkan val loss yang lebih rendah dibandingkan 1 layer. Hal ini menunjukkan bahwa LSTM mampu memanfaatkan kedalaman tambahan secara efektif karena mekanisme gerbangnya yang mengatasi vanishing gradient, sehingga layer kedua memberikan kapasitas representasi tambahan yang bermanfaat.

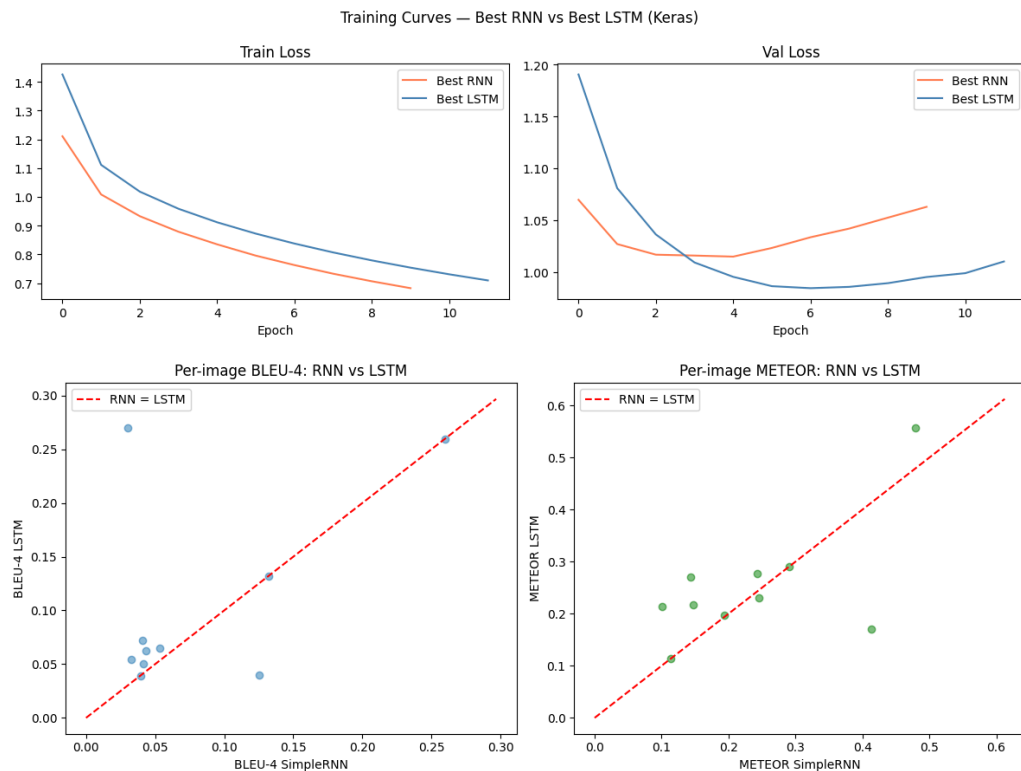
C) Perbandingan ukuran hidden state: RNN

Berdasarkan tabel evaluasi, hidden size H512 mencapai rata-rata BLEU-4 tertinggi sebesar 0.0735, sedangkan H128 dan H256 menghasilkan nilai yang hampir setara yaitu 0.0590 dan 0.0582. Dari sisi waktu eksekusi, H512 membutuhkan waktu lebih lama yaitu sekitar 17-22 detik dibandingkan H128 yang hanya sekitar 10-18 detik. Hal ini menunjukkan adanya trade-off antara kualitas caption dan efisiensi komputasi, di mana hidden size yang lebih besar memberikan kapasitas representasi yang lebih kaya namun membutuhkan waktu inferensi yang lebih panjang.

D) Perbandingan ukuran hidden state: LSTM

Pada LSTM, pola yang sama terlihat di mana H512 menghasilkan BLEU-4 tertinggi. Konfigurasi lstm_L2_H512 bahkan merupakan model terbaik secara keseluruhan dengan BLEU-4 sebesar 0.0914 dan METEOR 0.2465. Waktu inferensi LSTM H512 dengan 2 layer mencapai 72.27 detik yang merupakan yang terlama, namun menghasilkan kualitas caption terbaik. Sehingga pemilihan hidden size pada LSTM perlu mempertimbangkan keseimbangan antara kualitas caption yang dihasilkan dan sumber daya komputasi yang tersedia.

E) Perbandingan RNN vs LSTM

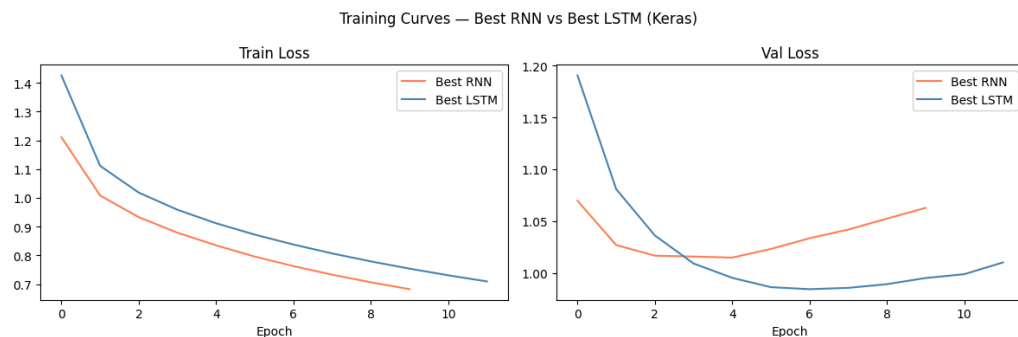


Metrik	SimpleRNN	LSTM
Mean BLEU-4	0.0798	0.1044
Mean METEOR	0.2371	0.2539
Avg time/image (ms)	395.8	136.6

Jika melihat grafik training dan validation loss, RNN mencapai train loss yang lebih rendah yaitu sekitar 0.68 namun val loss-nya mulai naik kembali setelah epoch 2 hingga mencapai sekitar 1.06, mengindikasikan RNN mengalami overfitting lebih cepat. Sebaliknya, LSTM mencapai val loss yang lebih rendah yaitu sekitar 0.98 dan tetap stabil hingga akhir pelatihan. Hal ini sejalan dengan hasil metrik di mana LSTM mengungguli RNN pada Mean BLEU-4 sebesar 0.1044 vs 0.0798 dan Mean METEOR sebesar 0.2539 vs 0.2371, meskipun LSTM juga lebih cepat per gambar yaitu 136.6ms dibandingkan RNN sebesar 395.8ms.

Dari scatter plot per-gambar, sebagian besar titik berada di atas garis diagonal pada grafik BLEU-4 yang berarti LSTM menghasilkan skor lebih tinggi untuk mayoritas gambar. Keunggulan LSTM ini berkaitan dengan mekanisme gerbangnya yang memungkinkan model mempertahankan informasi jangka panjang melalui cell state secara selektif, sehingga mampu menghasilkan caption yang lebih koheren dibandingkan RNN yang rentan terhadap vanishing gradient dan mudah kehilangan konteks dari awal sekuens.

F) Perbandingan Keras vs from scratch



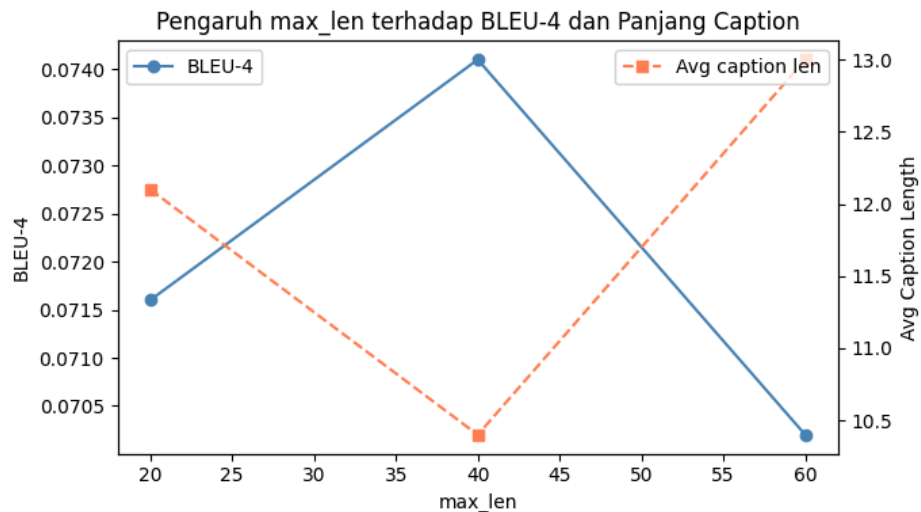
No.	Variasi	BLEU-4	METEOR	WAKTU
1.	RNN Keras (L1 & H512)	0.0576	0.2070	77.71
2.	RNN Scratch (L1 & H512)	0.0576	0.2070	0.71
3.	LSTM Keras (L2 & H512)	0.0809	0.2302	88.79
4.	LSTM Scratch (L2 & H512)	0.0809	0.2302	0.85

Berdasarkan tabel perbandingan, implementasi from scratch menghasilkan BLEU-4 dan METEOR yang identik dengan Keras yaitu 0.0576 dan 0.2070 untuk RNN, serta 0.0809 dan 0.2302 untuk LSTM. Hal ini dikonfirmasi oleh hasil prediksi yang 100% identik pada kedua arsitektur, membuktikan bahwa implementasi forward propagation from scratch telah mereplikasi operasi Keras secara akurat menggunakan bobot yang sama.

Perbedaan yang signifikan hanya terjadi pada waktu inferensi di mana from scratch jauh lebih cepat yaitu RNN 0.71 detik dan LSTM 0.85 detik untuk 100 sampel, dibandingkan Keras yang membutuhkan 77.71 detik dan 88.79 detik. Hal ini disebabkan oleh overhead kompilasi XLA dan inisialisasi GPU pada Keras yang sangat dominan untuk batch kecil, sedangkan implementasi NumPy murni langsung mengeksekusi

operasi matrix tanpa overhead tersebut. Karena kedua implementasi menggunakan bobot yang sama, grafik training dan validation loss juga identik.

G) Pengaruh panjang maksimum caption terhadap BLEU-4



Berdasarkan grafik pengaruh max_len terhadap BLEU-4 dan panjang caption, terdapat tiga variasi yang diujikan dengan hasil sebagai berikut.

a. Pendek (20)

Menghasilkan BLEU-4 sebesar 0.0716 dengan rata-rata panjang caption 12.1 kata. Meskipun model belum mencapai batas maksimalnya, ruang gerak yang terbatas membuat model tidak leluasa menghasilkan deskripsi yang lengkap sehingga banyak n-gram relevan yang tidak terbentuk.

b. Optimal (40)

Menghasilkan BLEU-4 tertinggi sebesar 0.0741 dengan rata-rata panjang caption 10.4 kata yang merupakan yang terpendek di antara ketiga variasi. Hal ini menunjukkan bahwa dengan max_len=40, model berhenti secara natural melalui token end tanpa dipotong paksa, dan panjang caption yang dihasilkan paling mendekati distribusi panjang

ground truth pada dataset Flickr8k.

c. Panjang (60)

Menghasilkan BLEU-4 terendah sebesar 0.0702 dengan rata-rata panjang caption 13.0 kata. Meskipun model diberikan ruang yang lebih besar, justru menghasilkan performa terburuk karena model cenderung menambahkan kata-kata yang tidak relevan setelah deskripsi inti selesai, sehingga menurunkan presisi BLEU-4.

Dari ketiga variasi tersebut dapat disimpulkan bahwa pemilihan `max_len` yang terlalu kecil maupun terlalu besar sama-sama berdampak negatif terhadap kualitas caption, dan nilai optimal adalah yang paling sesuai dengan distribusi panjang caption ground truth pada dataset yang digunakan.

IV. Kesimpulan dan Saran

A. Kesimpulan

Berdasarkan hasil implementasi dan eksperimen pada model Convolutional Neural Network (CNN), Simple Recurrent Neural Network (Simple RNN), dan Long Short-Term Memory (LSTM), dapat disimpulkan bahwa:

1. Implementasi forward propagation from scratch berhasil mereplikasi hasil Keras secara identik pada kedua model, baik CNN dengan F1-macro 0.8536 maupun RNN dan LSTM dengan BLEU-4 dan METEOR yang sama persis, membuktikan bahwa implementasi operasi konvolusi, pooling, embedding, dan sel RNN/LSTM menggunakan NumPy telah dilakukan secara akurat.
2. Pada tugas klasifikasi gambar menggunakan dataset Intel Image Classification, model CNN dengan jumlah layer konvolusi lebih banyak, filter lebih kecil, kernel size 3x3, dan max pooling menghasilkan performa terbaik dengan F1-macro 0.8536. Parameter sharing pada Conv2D terbukti lebih unggul dibandingkan LocallyConnected2D karena berperan sebagai regularisasi implisit yang menghasilkan generalisasi lebih baik meskipun menggunakan parameter jauh lebih sedikit.
3. Pada tugas image captioning menggunakan dataset Flickr8k, LSTM secara konsisten mengungguli Simple RNN pada metrik BLEU-4 maupun METEOR, dengan konfigurasi terbaik lstm_L2_H512 mencapai BLEU-4 sebesar 0.0914. Keunggulan LSTM disebabkan oleh mekanisme gerbang yang mampu mengatasi vanishing gradient dan mempertahankan informasi jangka panjang, sementara RNN lebih mudah kehilangan konteks pada sekuens yang panjang. Hidden size yang lebih besar secara konsisten menghasilkan kualitas caption yang lebih baik pada kedua arsitektur, meskipun dengan konsekuensi waktu inferensi yang lebih lama.

B. Saran

1. Untuk pengembangan model CNN lebih lanjut, disarankan untuk mengeksplorasi variasi arsitektur yang lebih beragam seperti penambahan dropout dan batch normalization pada setiap layer konvolusi agar model lebih tahan terhadap overfitting, serta mencoba rentang hyperparameter yang lebih luas untuk menemukan konfigurasi optimal yang lebih baik.
2. Pada model image captioning, disarankan untuk mengeksplorasi mekanisme attention yang memungkinkan decoder memfokuskan perhatian pada

bagian gambar yang relevan untuk setiap kata yang dihasilkan, sehingga dapat meningkatkan kualitas caption secara signifikan dibandingkan arsitektur inject yang hanya menggunakan satu vektor fitur global.

3. Untuk implementasi from scratch, disarankan untuk meningkatkan efisiensi komputasi dengan memanfaatkan vektorisasi penuh dan operasi batch yang lebih optimal agar waktu inferensi mendekati performa Keras, serta mempertimbangkan penggunaan operasi NumPy yang lebih efisien seperti einsum untuk menggantikan perkalian matriks bertingkat.

V. Pembagian Tugas

No	Mahasiswa	NIM	Tugas
1.	Stanislaus Ardy Bramantyo	18223057	- Mengerjakan Laporan - Membuat Notebook CNN & RNN/LSTM
2.	Nazwan Siddqi Muttaqin	18223066	- Mengerjakan Laporan - Melakukan training model - Membuat implementasi CNN - Memfinalisasi Notebook
3.	Matthew Sebastian Kurniawan	18223096	- Mengerjakan Laporan - Membuat implementasi RNN & LSTM - Mengimplementasikan backward propagation - Mengimplementasikan beam search decoder - Mengimplementasikan init-inject captioning - Mengimplementasikan visualisasi feature map CNN dan Grad-CAM

VI. Referensi

- Alescontrela. (n.d.). "Numpy-CNN/CNN/backward.py." Diakses pada 10 Mei 2026. <https://github.com/Alescontrela/Numpy-CNN/blob/master/CNN/backward.py>
- Basajay, D. (n.d.). "Image-Caption-Generator: A neural network to generate captions for an image using CNN and RNN with BEAM Search." Diakses pada 10 Mei 2026. <https://github.com/dabasajay/Image-Caption-Generator>
- Dtransposed. (2018). "LSTM neural network: A closer look under the hood." Diakses pada 10 Mei 2026. <https://dtransposed.github.io/blog/2018/05/19/Under-the-hood-LSTM/>
- Hansen, N. (n.d.). "rnn_lstm_from_scratch: How to build RNNs and LSTMs from scratch with NumPy." Diakses pada 10 Mei 2026. https://github.com/nicklashansen/rnn_lstm_from_scratch
- Hochreiter, S., & Schmidhuber, J. (1997). "Long short-term memory." Diakses pada 10 Mei 2026. <https://www.bioinf.jku.at/publications/older/2604.pdf>
- Moeinh77. (n.d.). "Image-Captioning-with-Beam-Search: Generating image captions using Xception Network and Beam Search in Keras." Diakses pada 10 Mei 2026. <https://github.com/Moeinh77/Image-Captioning-with-Beam-Search>
- Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). "BLEU: A method for automatic evaluation of machine translation." Diakses pada 10 Mei 2026. <https://aclanthology.org/P02-1040.pdf>
- Sami, Z. (n.d.). "CNN-from-Scratch: A scratch implementation of Convolutional Neural Network in Python using only numpy." Diakses pada 10 Mei 2026. <https://github.com/zishansami102/CNN-from-Scratch>
- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). "Grad-CAM: Visual explanations from deep networks via gradient-based localization." Diakses pada 10 Mei 2026. <https://arxiv.org/abs/1610.02391>
- Tanti, M., Gatt, A., & Camilleri, K. P. (2017). "Where to put the image in an image caption generator." Diakses pada 10 Mei 2026. <https://arxiv.org/abs/1703.09137>
- Vinyals, O., Toshev, A., Bengio, S., & Erhan, D. (2015). "Show and tell: A neural image caption generator." Diakses pada 10 Mei 2026. <https://arxiv.org/abs/1411.4555>
- You359. (n.d.). "Keras-CNNVisualization: Understanding of CNN with various visualization techniques." Diakses pada 10 Mei 2026. <https://github.com/you359/Keras-CNNVisualization>

VII. Lampiran

FORMAT DEKLARASI PENGGUNAAN AI SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Stanislaus Ardy Bramantyo/18223057, Nazwan Siddqi Muttaqin/18223066,
Matthew Sebastian Kurniawan/18223096

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi

Kode Mata Kuliah : IF3270

Nama Mata Kuliah : Pembelajaran Mesin

Judul Tugas/Proposal : Tugas Besar 2 Convolutional Neural Network dan Recurrent Neural Network

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, DeepL, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Rendah. Untuk memastikan keselarasan teks satu dengan yang lainnya.
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Medium. Digunakan sebagai teman diskusi dan menganalisis sesuatu.
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 13-03-2026



Stanislaus Ardy Bramantyo
18223057



Nazwan Siddqi Muttaqin
18223066



Matthew Sebastian Kurniawan
18223096